University of Ottawa
CSI 2110 - Data Structures and Algorithms
Laboratory 4
Sorting Algorithms, Divide-and-Conquer, Heap-Sort, and String Pattern Matching

Professor Mohamed Ali Ibrahim, PhD, PEng

Term Spring/Summer 2026

Lab Statement Available	Monday, June 29, 2026
Lab Period	Monday, June 29, 2026 to Saturday, July 11, 2026
First Lab Session	Friday, July 3, 2026
Submission	Brightspace, by Saturday, July 11, 2026, 11:59 PM
Programming Language	Java

1. Laboratory Overview

The purpose of this laboratory is to help students implement, test, and compare important sorting algorithms and basic string pattern-matching algorithms. This lab is aligned with the topics covered in Chapter 12 and Chapter 13 of the course textbook.

The focus is on divide-and-conquer sorting, comparison-based sorting, heap-sort, and string searching. Students will implement several algorithms, analyze their time complexity, test their behavior on different inputs, and compare their custom implementations with Java library methods.

- Merge-sort and the divide-and-conquer strategy
- Quick-sort and pivot selection
- Heap-sort using a heap-based approach
- Comparison of sorting algorithms
- Best-case, average-case, and worst-case behavior
- Stability and in-place sorting concepts
- Basic string pattern matching
- Brute-force pattern matching
- Java implementation, testing, complexity analysis, and reporting

2. Topics Covered in Lab 4

Chapter Area	Topics Included in Lab 4
Chapter 12 - Sorting and Selection	Merge-sort; divide-and-conquer; quick-sort; heap-sort; sorting comparison; stability; in-place sorting; time complexity of sorting algorithms.
Chapter 13 - Text Processing	Strings; pattern matching; brute-force pattern matching; pattern occurrence counting; searching inside text; algorithmic efficiency for text processing.

3. Learning Objectives

- Explain the divide-and-conquer strategy used by merge-sort.
- Implement merge-sort in Java.
- Implement quick-sort and explain the role of the pivot.
- Implement heap-sort or use a custom heap-based approach to sort data.
- Compare merge-sort, quick-sort, heap-sort, and simple sorting algorithms.

- Explain best-case, average-case, and worst-case running times.
- Distinguish between stable and unstable sorting algorithms.
- Distinguish between in-place and not-in-place sorting algorithms.
- Implement a brute-force string pattern-matching algorithm.
- Count and report all occurrences of a pattern inside a text.
- Test algorithms using several datasets and edge cases.
- Prepare a short technical report explaining implementation choices, tests, and complexity trade-offs.

4. Problem Description

Students will design and implement a small Course Data Analyzer. The system models a simplified academic data-processing tool that sorts student records, ranks assignment submissions, compares sorting algorithms, and searches for keywords or patterns in text comments.

The lab is divided into required parts. Each part may be implemented in a separate class, but all components must be demonstrated through a complete Main.java test program. The goal is not only to make the code work, but also to explain the algorithmic choices and the efficiency of each operation.

5. Required Data Models

Students must define simple data classes similar to the following. Additional fields may be added if they are clearly explained in the report.

```
public class StudentRecord {
    private int studentNumber;
    private String name;
    private double grade;
    private int submissionTime; // for example, minutes after deadline or timestamp order

    // constructors, getters, setters, and toString()
}

public class TextDocument {
    private int id;
    private String title;
    private String content;

    // constructors, getters, setters, and toString()
}
```

6. Part A - Merge-Sort and Divide-and-Conquer

Implement merge-sort for sorting an array or ArrayList of StudentRecord objects.

Requirements

- Implement a MergeSort class.
- Sort StudentRecord objects by grade in ascending or descending order.
- Clearly divide the algorithm into divide, conquer, and combine steps.
- Implement a merge method that combines two sorted sublists.
- Demonstrate merge-sort on at least 10 StudentRecord objects.
- Print the list before and after sorting.
- Explain why merge-sort is a divide-and-conquer algorithm.
- Explain the time complexity of merge-sort.

Expected behavior example:

```
Before sorting by grade:
1004 - Amina - 72.5
1002 - Karim - 91.0
1007 - Sara - 84.0

After sorting by grade:
1004 - Amina - 72.5
1007 - Sara - 84.0
1002 - Karim - 91.0
```

7. Part B - Quick-Sort

Implement quick-sort for sorting StudentRecord objects.

Requirements

- Implement a QuickSort class.
- Choose and clearly document a pivot strategy.
- The pivot may be the first element, last element, middle element, or a randomly selected element.
- Implement partitioning.
- Sort StudentRecord objects by studentNumber or grade.
- Demonstrate quick-sort on at least 10 StudentRecord objects.
- Print the list before and after sorting.
- Explain how the pivot affects the performance of quick-sort.
- Explain the best-case, average-case, and worst-case time complexity of quick-sort.
- Include at least one test case where the input is already sorted and one test case where the input is reverse sorted.

8. Part C - Heap-Sort

Implement heap-sort using an array-based heap approach.

Requirements

- Implement a HeapSort class.
- Use an array or ArrayList representation.
- Build a heap from the input data.
- Use the heap to repeatedly remove the maximum or minimum element.
- Sort StudentRecord objects by grade or submissionTime.
- Demonstrate heap-sort on at least 10 StudentRecord objects.
- Print the internal heap or key heap steps when possible.
- Explain why heap-sort has O(n log n) running time.
- Explain whether heap-sort is stable or unstable.
- Explain whether heap-sort is in-place in the implementation used.
- Students may reuse ideas from the heap-based priority queue studied previously, but the sorting logic must be clearly implemented and demonstrated.

9. Part D - Sorting Algorithm Comparison

Students must compare at least four sorting algorithms: merge-sort, quick-sort, heap-sort, and one simple sorting algorithm such as selection sort or insertion sort.

Requirements

- Implement or reuse a simple sorting algorithm for comparison.
- Run all sorting algorithms on the same datasets.
- Test at least three input cases: random order, already sorted, and reverse sorted.
- Use at least three dataset sizes, for example 10 records, 100 records, and 1000 records.
- Count the number of comparisons or measure running time.
- Present the results in a table in the report.
- Explain which algorithm performed best in each case and why.

Suggested comparison table:

Dataset Type	Size	Merge-Sort Time	Quick-Sort Time	Heap-Sort Time	Simple Sort Time
Random	10				
Random	100				
Random	1000				
Sorted	10				
Sorted	100				
Sorted	1000				
Reverse Sorted	10				
Reverse Sorted	100				
Reverse Sorted	1000				

10. Part E - String Pattern Matching

Implement a brute-force string pattern-matching algorithm.

Required operations

Operation	Description
findFirst(text, pattern)	Return the index of the first occurrence of the pattern, or -1 if the pattern is not found.
findAll(text, pattern)	Return or print all starting indices where the pattern occurs.
countOccurrences(text, pattern)	Return the number of times the pattern occurs in the text.
containsPattern(text, pattern)	Return true if the pattern exists in the text, otherwise false.

Requirements

- Implement a PatternMatcher class.
- Use a brute-force pattern-matching approach.
- Search for a pattern inside the content of TextDocument objects.
- Demonstrate the algorithm on at least 3 TextDocument objects.
- Test patterns that appear once, multiple times, and not at all.
- Test edge cases such as empty text, empty pattern, pattern longer than the text, pattern equal to the text, and overlapping patterns.

Expected behavior example:

```
Text: "data structures and data algorithms"
Pattern: "data"
findFirst returns 0
```

```
findAll returns [0, 20]
countOccurrences returns 2
```

11. Part F - Java Collections and Library Comparison

Students must compare their custom implementations with Java library methods.

Custom Implementation	Suggested Java Library Comparison
Custom merge-sort	Collections.sort or Arrays.sort
Custom quick-sort	Arrays.sort behavior discussion
Custom heap-sort	PriorityQueue-based sorting discussion
Custom simple sort	Collections.sort or Arrays.sort
Custom pattern matcher	String.indexOf, String.contains, or regular expressions

In the report, students must explain at least two similarities and two differences between their custom implementations and the selected Java library methods. The comparison should focus on correctness, efficiency, ease of use, stability, in-place behavior, and practical use in real software.

12. Part G - Complexity Analysis

Students must complete a written complexity analysis section in the report. The analysis must include Big-O notation and short justifications.

Required Complexity Table

Algorithm / Operation	Best Case	Average Case	Worst Case	Space Complexity	Stable?	In-Place?
Merge-sort						
Quick-sort						
Heap-sort						
Insertion sort or selection sort						
Brute-force pattern matching						

Required Written Questions

- Explain why merge-sort is considered a divide-and-conquer algorithm.
- Explain why merge-sort has O(n log n) running time.
- Explain how quick-sort partitions the input.
- Explain how pivot selection can affect quick-sort performance.
- Explain why heap-sort has O(n log n) running time.
- Compare merge-sort, quick-sort, and heap-sort in terms of stability and memory usage.
- Explain why simple sorting algorithms may still be useful for small inputs.
- Explain how brute-force pattern matching works.
- Explain the worst-case time complexity of brute-force pattern matching.
- Compare String.indexOf with a custom brute-force pattern-matching implementation.

13. Testing Requirements

Students must create a Main.java file that demonstrates the correctness of all required implementations and exercises. The test program must include:

- Creating at least 10 StudentRecord objects.
- Creating at least 3 TextDocument objects.
- Testing merge-sort on StudentRecord objects.
- Testing quick-sort on StudentRecord objects.
- Testing heap-sort on StudentRecord objects.
- Testing one simple sorting algorithm.
- Testing sorting by at least two different keys, such as grade and studentNumber.
- Testing random, sorted, and reverse-sorted inputs.
- Testing brute-force pattern matching.
- Testing patterns that appear once, multiple times, and not at all.
- Testing edge cases such as empty input, one-element input, duplicate grades, and missing patterns.
- Printing clear output that shows the result of each test.

14. Deliverables

Each group must submit a ZIP file on Brightspace containing:

- StudentRecord.java
- TextDocument.java
- MergeSort.java
- QuickSort.java
- HeapSort.java
- SimpleSort.java
- PatternMatcher.java
- Main.java
- A short lab report in PDF format

The report should include:

- Team members names and student numbers
- Description of the implementation and design choices
- Screenshots or copied output from test runs
- Sorting comparison table
- Complexity analysis table
- Answers to the required written questions
- Short discussion comparing custom implementations with Java library methods
- Problems encountered and how they were solved

15. Laboratory Demonstration Requirement

The laboratory demonstration is a mandatory component of Lab 4. Each team must demonstrate its implementation to the Teaching Assistant (TA).

- Each team must complete a demonstration with the TA for the lab report to be considered for grading.
- All team members must be present during the demonstration.
- Each member may be asked to explain the implementation, tests, sorting behavior, pattern-matching behavior, and complexity analysis.

- The demonstration should clearly show that the submitted code runs correctly and matches the requirements of this lab.
- If a team does not complete the laboratory demonstration with the TA, the maximum mark the team can receive for the lab is 50%. This maximum would only apply to an exceptionally strong and complete written report. In general, without the required demonstration, the final lab grade is expected to be significantly lower than 50%.

16. Grading Rubric

Total: 100 points

Component	Points
Merge-sort implementation and divide-and-conquer explanation	18
Quick-sort implementation and pivot/partition explanation	16
Heap-sort implementation and heap-based sorting explanation	16
Sorting algorithm comparison and experimental results	15
String pattern-matching implementation	15
Java library comparison	5
Testing, edge cases, and demonstration quality	7
Complexity analysis and written justification questions	6
Code quality, comments, organization, and report quality	2
Total	100

17. Important Notes

- Late submissions are not accepted unless an official extension has been granted by the professor.
- All team members must understand the submitted code.
- Students may be asked to explain their implementation during the lab or tutorial.
- Code must be written in Java.
- Plagiarism or copied code will receive a grade of zero and may be reported according to university academic integrity policies.
- Students may use Java library sorting methods only in Part F for comparison, unless explicitly stated otherwise.
- The required custom algorithms must be implemented by the students.

18. Timeline

Date	Work
Monday, June 29	Lab statement becomes available. Read the full statement and review merge-sort, quick-sort, heap-sort, and basic pattern matching.
June 30 - July 1	Create the Java project structure. Begin StudentRecord.java, TextDocument.java, and the testing framework in Main.java.
Thursday, July 2	Review Sorting I: merge-sort and divide-and-conquer. Implement MergeSort.java and test it on at least 10 records.
Friday, July 3	First lab session. Students should begin implementing and testing MergeSort.java and prepare questions about divide-and-conquer sorting.
July 4 - July 6	Complete merge-sort testing. Begin QuickSort.java. Implement pivot selection and partitioning. Test quick-sort on several datasets.

Tuesday, July 7	Review Sorting II: quick-sort, heap-sort, and sorting comparison. Complete QuickSort.java and begin HeapSort.java.
July 8	Complete HeapSort.java and implement one simple sorting algorithm for comparison.
Thursday, July 9	Review strings and pattern matching. Implement PatternMatcher.java and test findFirst, findAll, countOccurrences, and containsPattern.
July 10	Complete sorting comparison experiments, Java library comparison, complexity analysis, and written questions.
Saturday, July 11	Submit the ZIP file and laboratory report on Brightspace by 11:59 PM.